

```
package Data;

import Data.Queue;
import Data.Node;

public class QueueUtils {

    public static <T> void iterateQueues(Queue<T> queue) {
        Node<T> current = queue.first;
        while (current != null) {
            System.out.println(current.GetInfo());
            current = current.GetNext();
        }
    }

    public static <T> T getFirstItem(Queue<T> queue) {
        return (queue.first != null) ? queue.first.GetInfo() : null;
    }

    public static <T> T getLastItem(Queue<T> queue) {
        return (queue.last != null) ? queue.last.GetInfo() : null;
    }

    public static <T> void rotateLeft(Queue<T> queue) {
        if (queue.first == null || queue.first.GetNext() == null)
            return;
        Node<T> oldFirst = queue.first;
        queue.first = oldFirst.GetNext();
        queue.last.SetNext(oldFirst);
        oldFirst.SetNext(null);
        queue.last = oldFirst;
    }

    public static <T> void rotateRight(Queue<T> queue) {
        if (queue.first == null || queue.first.GetNext() == null)
            return;
        Node<T> current = queue.first;
        Node<T> prev = null;
        while (current.GetNext() != null) {
            prev = current;
            current = current.GetNext();
        }
        current.SetNext(queue.first);
        queue.first = current;
        prev.SetNext(null);
        queue.last = prev;
    }
}
```

```
public static <T> int getNodePosition(Queue<T> queue, T value) {
    int pos = 0;
    Node<T> current = queue.first;
    while (current != null) {
        if (current.GetInfo().equals(value))
            return pos;
        current = current.GetNext();
        pos++;
    }
    return -1;
}

public static <T> Node<T> getNodeByIndex(Queue<T> queue, int index) {
    int pos = 0;
    Node<T> current = queue.first;
    while (current != null) {
        if (pos == index)
            return current;
        current = current.GetNext();
        pos++;
    }
    return null;
}
```

```

public static <T> void insertAtHead(Queue<T> queue, T value) {
    Node<T> newNode = new Node<>(value, queue.first);
    queue.first = newNode;
    if (queue.last == null) {
        queue.last = newNode;
    }
}

public static <T> void insertAtTail(Queue<T> queue, T value) {
    Node<T> newNode = new Node<>(value);
    if (queue.first == null) {
        queue.first = newNode;
        queue.last = newNode;
    } else {
        queue.last.SetNext(newNode);
        queue.last = newNode;
    }
}

public static <T> void insertAtPosition(Queue<T> queue, int index, T value) {
    if (index == 0) {
        insertAtHead(queue, value);
        return;
    }
    Node<T> current = queue.first;
    int pos = 0;
    while (current != null && pos < index - 1) {
        current = current.GetNext();
        pos++;
    }
    if (current != null) {
        Node<T> newNode = new Node<>(value, current.GetNext());
        current.SetNext(newNode);
        if (newNode.GetNext() == null) {
            queue.last = newNode;
        }
    }
}

```

```

public static <T> void deleteFirst(Queue<T> queue) {
    if (queue.first == null)
        return;
    queue.first = queue.first.GetNext();
    if (queue.first == null) {
        queue.last = null;
    }
}

public static <T> void deleteLast(Queue<T> queue) {
    if (queue.first == null || queue.first.GetNext() == null) {
        queue.first = null;
        queue.last = null;
        return;
    }
    Node<T> current = queue.first;
    while (current.GetNext().GetNext() != null) {
        current = current.GetNext();
    }
    current.SetNext(null);
    queue.last = current;
}

public static <T> void deleteAtPosition(Queue<T> queue, int index) {
    if (queue.first == null)
        return;
    if (index == 0) {
        deleteFirst(queue);
        return;
    }
    Node<T> current = queue.first;
    int pos = 0;
    while (current.GetNext() != null && pos < index - 1) {
        current = current.GetNext();
        pos++;
    }
    if (current.GetNext() != null) {
        if (current.GetNext() == queue.last) {
            queue.last = current;
        }
        current.SetNext(current.GetNext().GetNext());
    }
}

```

```

public static <T> Node<T> findMiddleItem(Queue<T> queue) {
    if (queue.first == null)
        return null;
    Node<T> slow = queue.first, fast = queue.first;
    while (fast != null && fast.GetNext() != null) {
        slow = slow.GetNext();
        fast = fast.GetNext().GetNext();
    }
    return slow;
}

public static <T> void reverseList(Queue<T> queue) {
    Node<T> prev = null;
    Node<T> current = queue.first;
    queue.last = queue.first;
    while (current != null) {
        Node<T> next = current.GetNext();
        current.SetNext(prev);
        prev = current;
        current = next;
    }
    queue.first = prev;
}

public static <T> void iterateReverse(Queue<T> queue) {
    Stack<T> stack = new Stack<>();
    Node<T> current = queue.first;
    while (current != null) {
        stack.push(current.GetInfo());
        current = current.GetNext();
    }
    while (!stack.isEmpty()) {
        System.out.println(stack.pop());
    }
}

public static <T> int countItems(Queue<T> queue) {
    int count = 0;
    Node<T> current = queue.first;
    while (current != null) {
        count++;
        current = current.GetNext();
    }
    return count;
}

public static <T> boolean containsValue(Queue<T> queue, T value) {
    Node<T> current = queue.first;
    while (current != null) {
        if (current.GetInfo().equals(value))
            return true;
        current = current.GetNext();
    }
    return false;
}

```



```

public static <T> Queue<T> copyList(Queue<T> queue) {
    Queue<T> newQueue = new Queue<>();
    Node<T> current = queue.first;
    while (current != null) {
        newQueue.Insert(current.GetInfo());
        current = current.GetNext();
    }
    return newQueue;
}

public static <T> Queue<T> mergeTwoLists(Queue<T> q1, Queue<T> q2) {
    Queue<T> newQueue = copyList(q1);
    Node<T> current = q2.first;
    while (current != null) {
        newQueue.Insert(current.GetInfo());
        current = current.GetNext();
    }
    return newQueue;
}
}

```